Informe Laboratorio 4

Sección 2

Alumno Oseas Poveda Huerta
e-mail: oseas.poveda@mail.udp.cl

Noviembre de 2026

Índice

1. Descripción **3**

2. Actividades **3**
2.1. Configuración y análisis de SSH . . . 3
2.1.1. Implementación y configuración del servidor SSH . . . 3
2.1.2. Análisis de paquetes SSH en Wireshark . . . 3
2.2. Análisis de captura de tráfico antigua . . . 4
2.2.1. Análisis de los mecanismos criptográficos . . . 4
2.2.2. Recuperación del PIN y derivación de la clave AES . . . 4
2.2.3. Descifrado de las transacciones y análisis final . . . 4

3. Desarrollo de actividades **6**
3.1. Actividad 1 . . . 6
3.1.1. Dockerfile . . . 6
3.1.2. Configuración del servidor . . . 7
3.1.3. Explicación de los algoritmos utilizados . . . 8
3.1.4. Conexión cliente-servidor . . . 9
3.1.5. Análisis de tráfico . . . 11
3.2. Actividad 2 . . . 15
3.2.1. Identificación del paquete RSA_PIN . . . 15
3.2.2. Identificación de los paquetes AESGCM . . . 15
3.2.3. Recuperación del PIN y derivación de la clave AES . . . 16
3.2.4. Descifrar los mensajes . . . 18
3.2.5. Análisis de criptografía postcuántica (PQC) . . . 21

Conclusiones y comentarios **21**

4. Bibliografia **21**

1. Descripción

Usted trabaja en una empresa especializada en la transferencia segura de información, archivos y contenido digital. Antes de incorporarse al desarrollo y operación de los sistemas reales de la organización, se le solicita realizar una demostración técnica que permita evaluar sus conocimientos en redes, protocolos de comunicación y criptografía.

Para ello, deberá implementar y configurar un servidor SSH, definiendo los mecanismos criptográficos utilizados para proteger la comunicación. Posteriormente, deberá establecer una conexión con el servidor y capturar el tráfico de red utilizando Wireshark.

A partir del análisis de los paquetes capturados, deberá identificar qué información puede ser observada por un tercero que tuviera acceso al tráfico de red y determinar qué elementos de la comunicación están protegidos mediante cifrado y cuáles pueden ser visibles, con el objetivo de evaluar el nivel de seguridad proporcionado por el protocolo SSH.

Finalmente, se le proporcionará una captura de tráfico de red obtenida mediante Wireshark. En ella se encuentran diversos mensajes correspondientes a transacciones que han sido cifrados mediante RSA.

El objetivo de esta actividad es analizar la captura de tráfico, identificar los paquetes que contienen la información cifrada y desarrollar un programa en Python que permita recuperar el contenido original.

2. Actividades

2.1. Configuración y análisis de SSH

2.1.1. Implementación y configuración del servidor SSH

Generar un contenedor Docker que permita ejecutar un servidor SSH accesible desde el equipo cliente (su computador). Posteriormente, configure el servidor mediante un archivo `sshd_config`, en el cual deberá especificar los algoritmos criptográficos utilizados durante la comunicación SSH: `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Por último, documente la función que cumple cada algoritmo dentro del proceso de establecimiento y comunicación de una conexión SSH.

2.1.2. Análisis de paquetes SSH en Wireshark

Una vez completada la configuración del servidor SSH, deberá analizar el tráfico de red generado durante el establecimiento de la conexión utilizando Wireshark.

Para ello, antes de realizar la conexión SSH, inicie la captura de tráfico en la interfaz de red correspondiente. Posteriormente, establezca una conexión con el servidor utilizando su máquina como cliente y analice los paquetes intercambiados durante el proceso de conexión.

A partir del análisis realizado, responda la siguiente pregunta:

¿Qué información es posible identificar a partir de los paquetes capturados? Mencione y explique al menos tres elementos que puedan observarse en el tráfico de la conexión SSH.

2.2. Análisis de captura de tráfico antigua

2.2.1. Análisis de los mecanismos criptográficos

Se le solicita analizar una captura de tráfico de red antigua, en formato .pcap, que contiene transacciones financieras cifradas mediante una combinación de criptografía asimétrica y simétrica. Para realizar la actividad, se le proporcionarán la captura y las claves pública y privada RSA correspondientes. Deberá identificar los paquetes relevantes y analizar el mecanismo de cifrado utilizado, considerando que RSA se emplea para cifrar un PIN de cuatro dígitos mediante OAEP con SHA-256, mientras que AES-GCM se utiliza para proteger el contenido de las transacciones financieras.

2.2.2. Recuperación del PIN y derivación de la clave AES

El primer paquete relevante de la captura contiene la etiqueta RSA_PIN, seguida de una salt de 16 bytes y del PIN cifrado mediante RSA. Deberá identificar y extraer estos campos, utilizar la clave privada RSA proporcionada para recuperar el PIN y, posteriormente, derivar la clave AES de 256 bits mediante Scrypt. Para ello, deberá utilizar el PIN recuperado, la salt extraída de la captura y los siguientes parámetros: (n=2**14), (r=8) y (p=1). Explique la función de la salt y por qué es necesario utilizar los mismos parámetros para obtener la clave AES original.

2.2.3. Descifrado de las transacciones y análisis final

Una vez obtenida la clave AES, deberá identificar todos los paquetes que contienen transacciones cifradas mediante AES-GCM. Para cada paquete, deberá extraer el nonce de 12 bytes y utilizarlo junto con la clave derivada y el texto cifrado para recuperar los mensajes originales, verificando su autenticidad. Implemente el procedimiento completo en un script de Python y presente las transacciones recuperadas en su formato original, incluyendo el identificador, el remitente, el destinatario y el monto. Documente el procedimiento realizado e incluya evidencias de los resultados obtenidos.

```
[+] PIN recuperado mediante RSA
[+] PIN: 4827
[+] Transacciones cifradas encontradas: 40

========== TRANSACCIONES ==========

[Transacción 1]
TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923

[Transacción 2]
TRANSACCION COMPLETADA|ID=TRX-346837|DE=Catalina Vargas|A=Valentina Rojas|MONTO=58911

[Transacción 3]
TRANSACCION COMPLETADA|ID=TRX-412154|DE=Sebastian Munoz|A=Constanza Vidal|MONTO=485213
```

Figura 1: Ejemplo del resultado

Finalmente, responda la siguiente pregunta, justificando su respuesta: ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC (Criptografía post-cuántica)?

3. Desarrollo de actividades

3.1. Actividad 1

3.1.1. Dockerfile

Para esta actividad se creó una imagen Docker basada en Ubuntu 22.04 que instala el servidor OpenSSH, crea un usuario de prueba llamado estudiante, genera las claves del servidor con ssh-keygen -A y copia el archivo de configuración sshd_config personalizado dentro de la imagen. Además, el servidor se deja corriendo en primer plano con la opción -D (y -e para que los registros salgan por la salida de error), de modo que el contenedor no se cierre apenas se inicia.

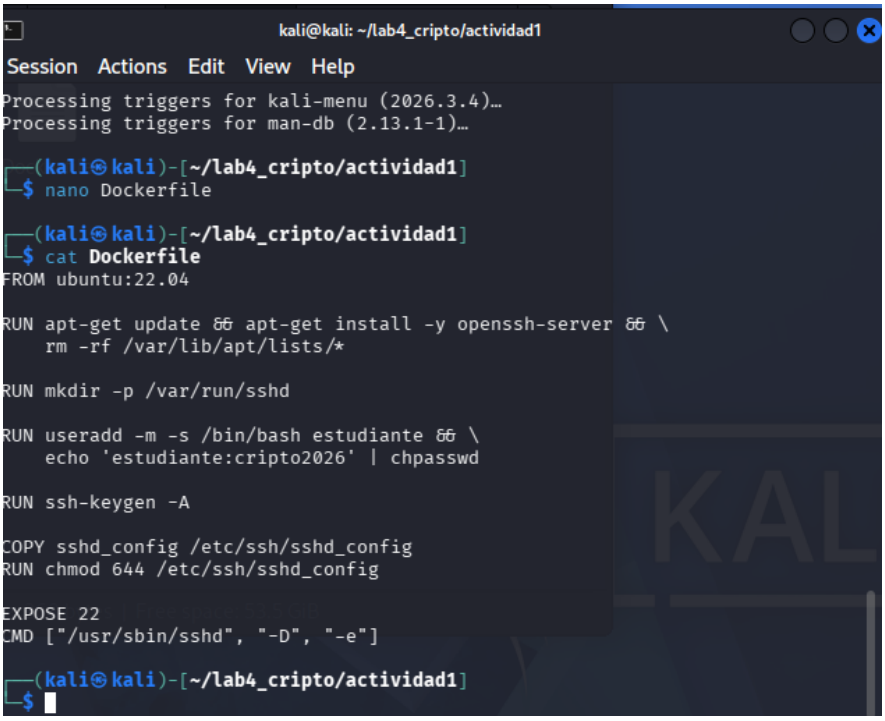


Figura 2: Contenido del archivo Dockerfile

La imagen se construyó con docker build y luego se levantó el contenedor mapeando el puerto 22 del contenedor al puerto 2222 del equipo, dejando el servidor accesible desde el computador cliente.

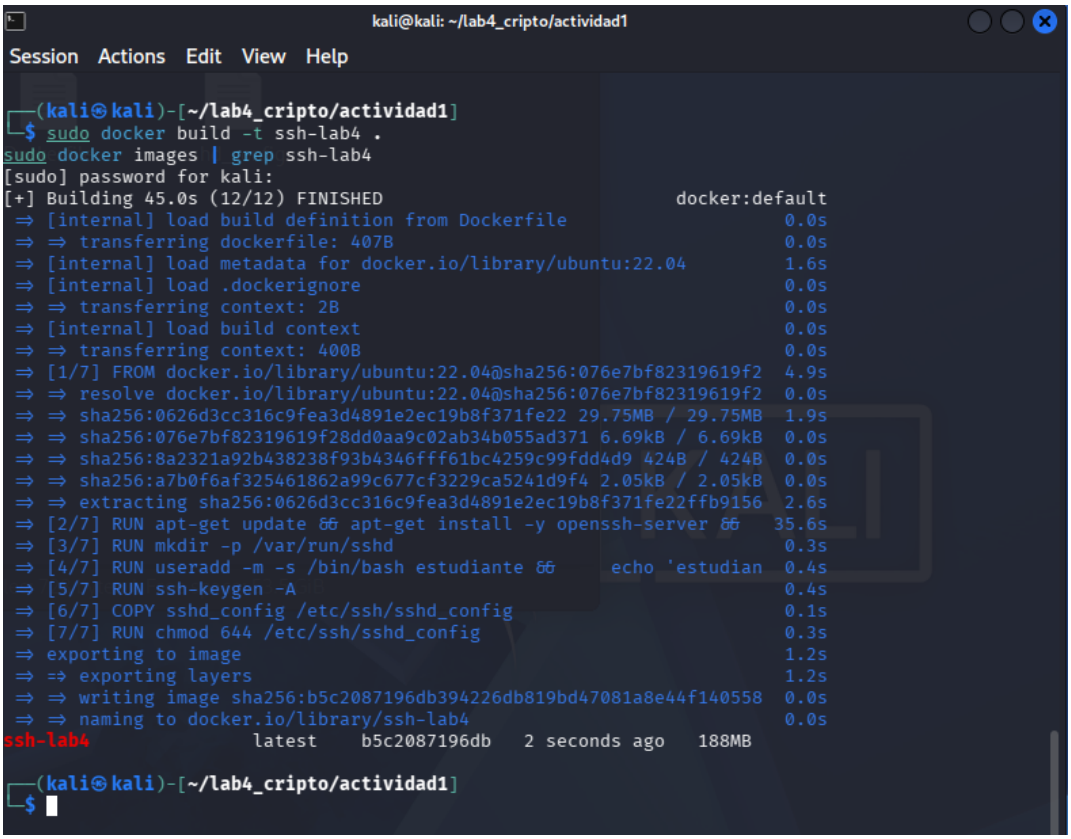


Figura 3: Construcción de la imagen ssh-lab4

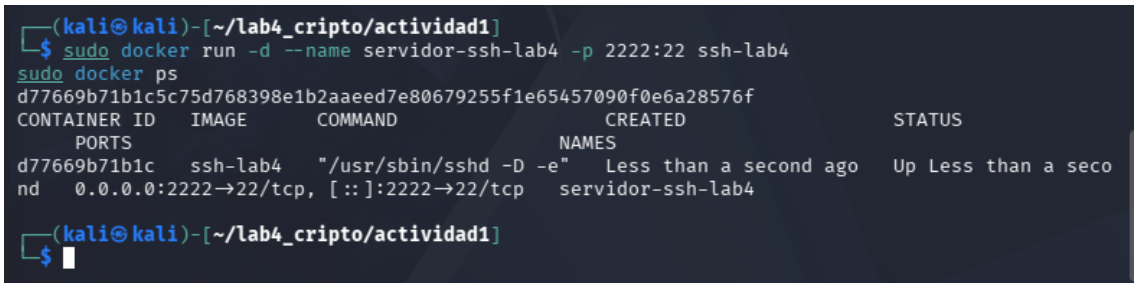


Figura 4: Contenedor servidor-ssh-lab4 en ejecucion

3.1.2. Configuración del servidor

En el archivo sshd config se indicaron los cuatro algoritmos solicitados usando las directivas KexAlgorithms, Ciphers, MACs y HostKeyAlgorithms. También se definió la clave de host ed25519 con la directiva HostKey y se restringieron las claves públicas aceptadas a ed25519, para que el servidor no ofrezca ningún otro algoritmo durante la negociación.

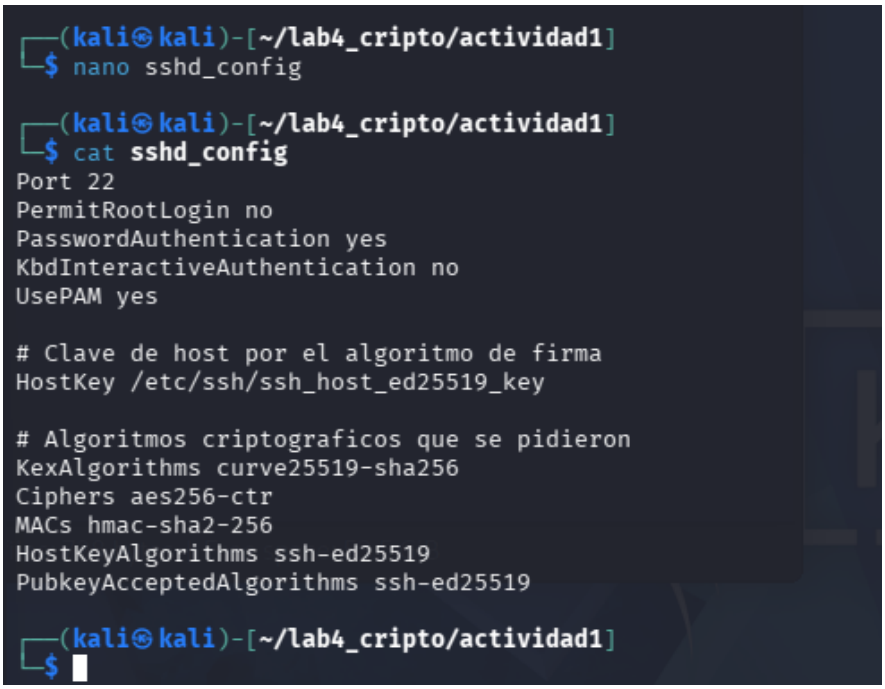


Figura 5: Contenido del archivo sshd config

Para comprobar que el servidor realmente está usando esta configuración, se ejecutó el comando sshd -T dentro del contenedor, filtrando solo las directivas de interés. En la salida se observa que cada una de ellas quedó con un único algoritmo.



Figura 6: Algoritmos activos en el servidor.

3.1.3. Explicación de los algoritmos utilizados

A continuación se resume la función que cumple cada algoritmo dentro del establecimiento y la protección de la conexión SSH.

Algoritmo	Función
`curve25519-sha256`	Intercambio de claves (ECDH sobre la curva Curve25519). Permite que cliente y servidor calculen una clave secreta común sin enviarla por la red. SHA-256 se usa para calcular el hash del intercambio y derivar las claves de la sesión.
`ssh-ed25519`	Firma digital del servidor. El servidor firma el hash del intercambio con su clave privada para demostrar su identidad al cliente y evitar ataques de tipo Man-in-the-Middle.
`aes256-ctr`	Cifrado simétrico de los datos. Una vez terminado el intercambio de claves, cifra toda la información de la sesión (contraseña, comandos y respuestas) con una clave de 256 bits.
`hmac-sha2-256`	Integridad y autenticidad. Agrega un código de autenticación a cada paquete para detectar si fue modificado o si se inyectaron datos durante el envío.

Tabla 1: Explicación de la función de cada algoritmo.

Cabe aclarar que, como el intercambio de claves usa claves efímeras, se obtiene forward secrecy si en el futuro se filtrara la clave privada del servidor, igualmente no se podrían descifrar sesiones pasadas que hayan sido capturadas.

3.1.4. Conexión cliente-servidor

Desde el cliente que es una máquina Kali Linux se realizó la conexión con ssh -v -p 2222 estudiante@localhost. En la salida se observa que se usaron los cuatro algoritmos configurados, que el usuario se autenticó correctamente con contraseña y que la sesión se abrió dentro del contenedor. En las siguientes figuras no se muestra la ejecución completa, sino solo las partes más relevantes.

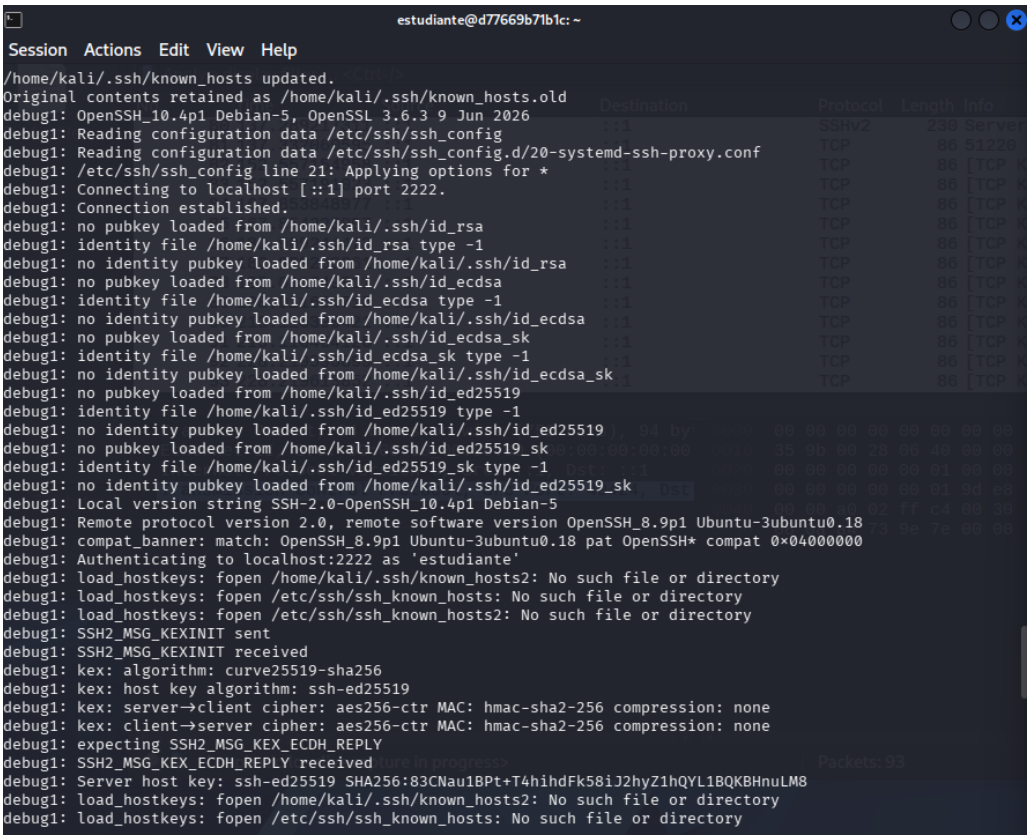


Figura 7: Conexión SSH negociación de algoritmos.

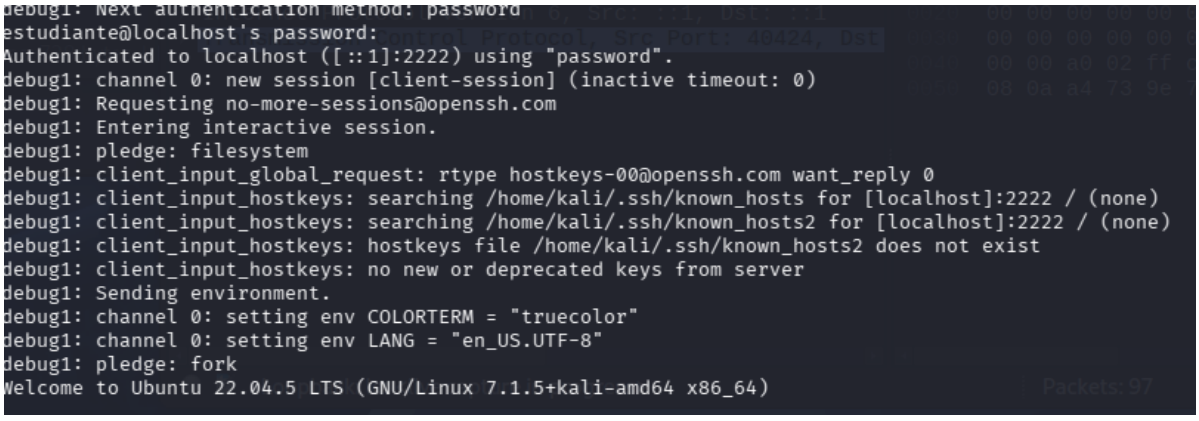


Figura 8: Conexión SSH autenticación.

En la sesión se ejecutaron los comandos whoami y hostname. El nombre de host que aparece en el prompt corresponde al ID del contenedor, lo que confirma que la sesión se abrió dentro de él y no en el equipo cliente.

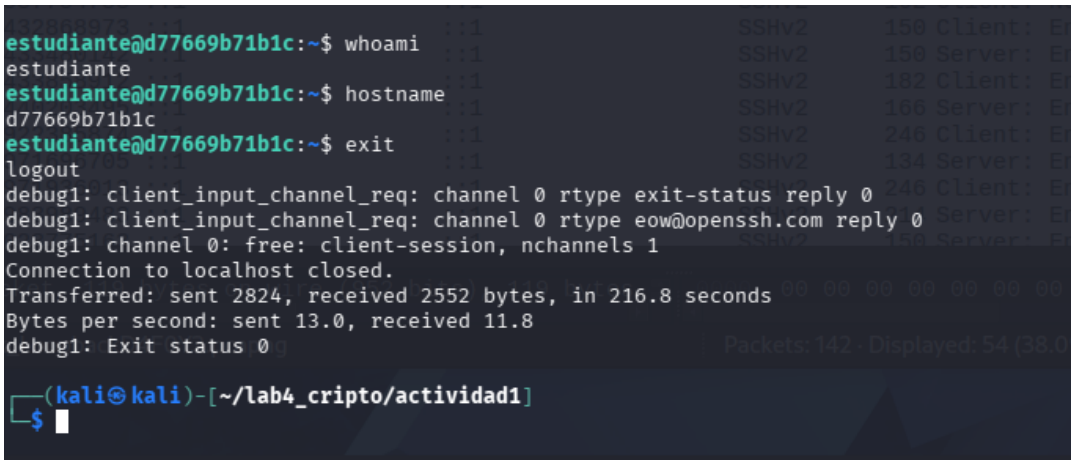


Figura 9: Conexión SSH sesión dentro del contenedor.

3.1.5. Análisis de tráfico

Antes de realizar la conexión se inició la captura de Wireshark en la interfaz de loopback, ya que el servidor se encuentra en el mismo equipo y se accede mediante localhost. Luego de conectarse y cerrar la sesión se detuvo la captura y se filtró por ssh && tcp.port == 2222, obteniendo el siguiente flujo:

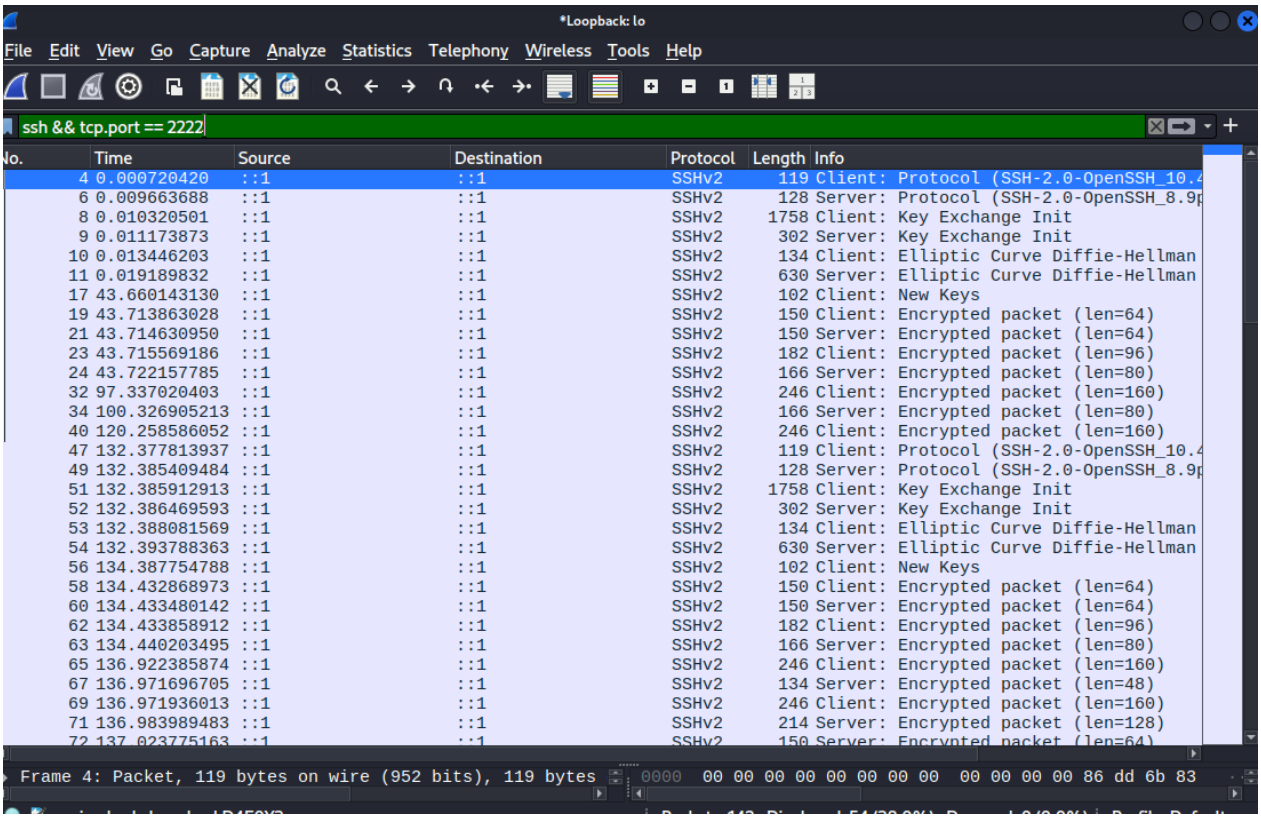


Figura 10: Paquetes de la conexión SSH.

En el flujo se distinguen primero los paquetes con la versión del protocolo (banner), luego el Key Exchange Init de cliente y servidor, el intercambio Elliptic Curve Diffie-Hellman con el mensaje New Keys, y finalmente los paquetes que ya aparecen como cifrados (Encrypted packet). En el paquete Key Exchange Init del servidor se pueden ver las listas de algoritmos que ofrece:

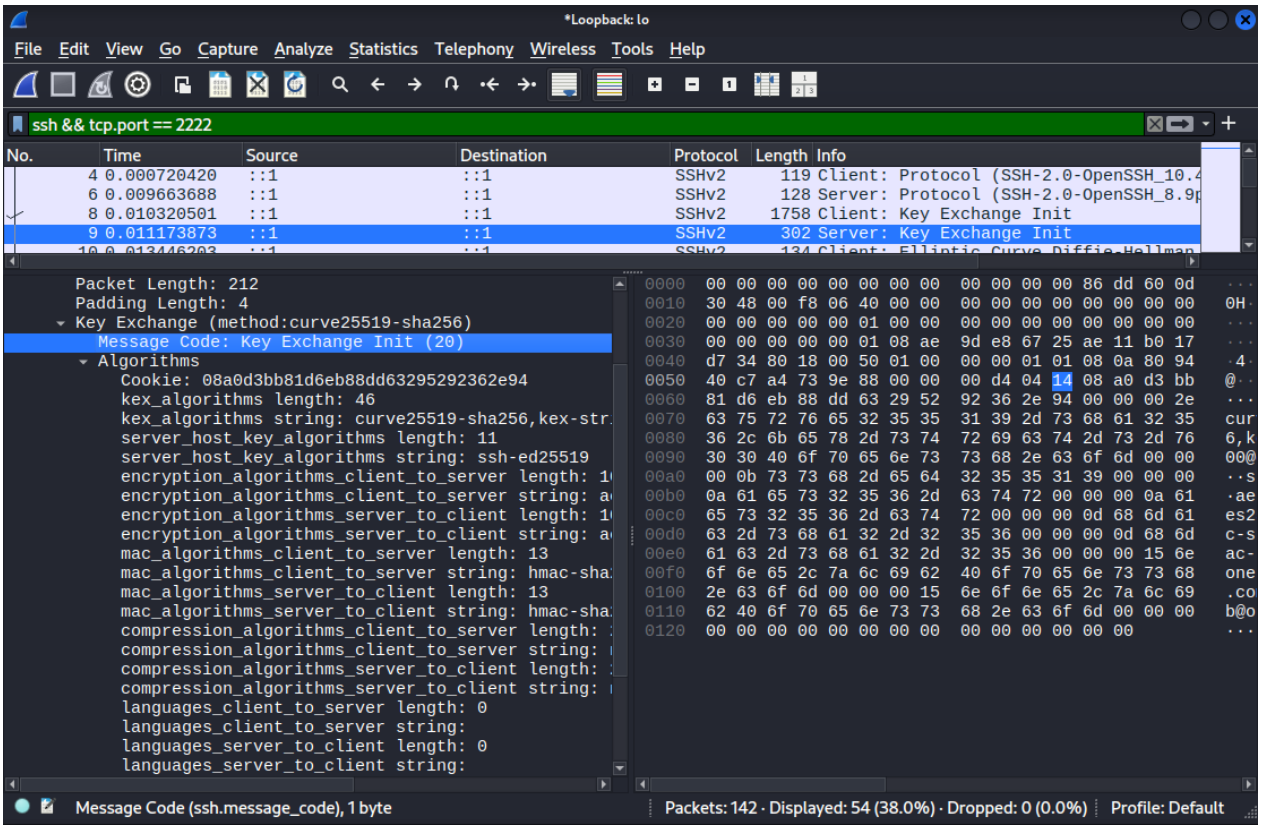


Figura 11: Paquete key Exchange Init del servidor

Y en el paquete ECDH Key Exchange Reply se observan la clave pública efímera del servidor, su clave de host y la firma:

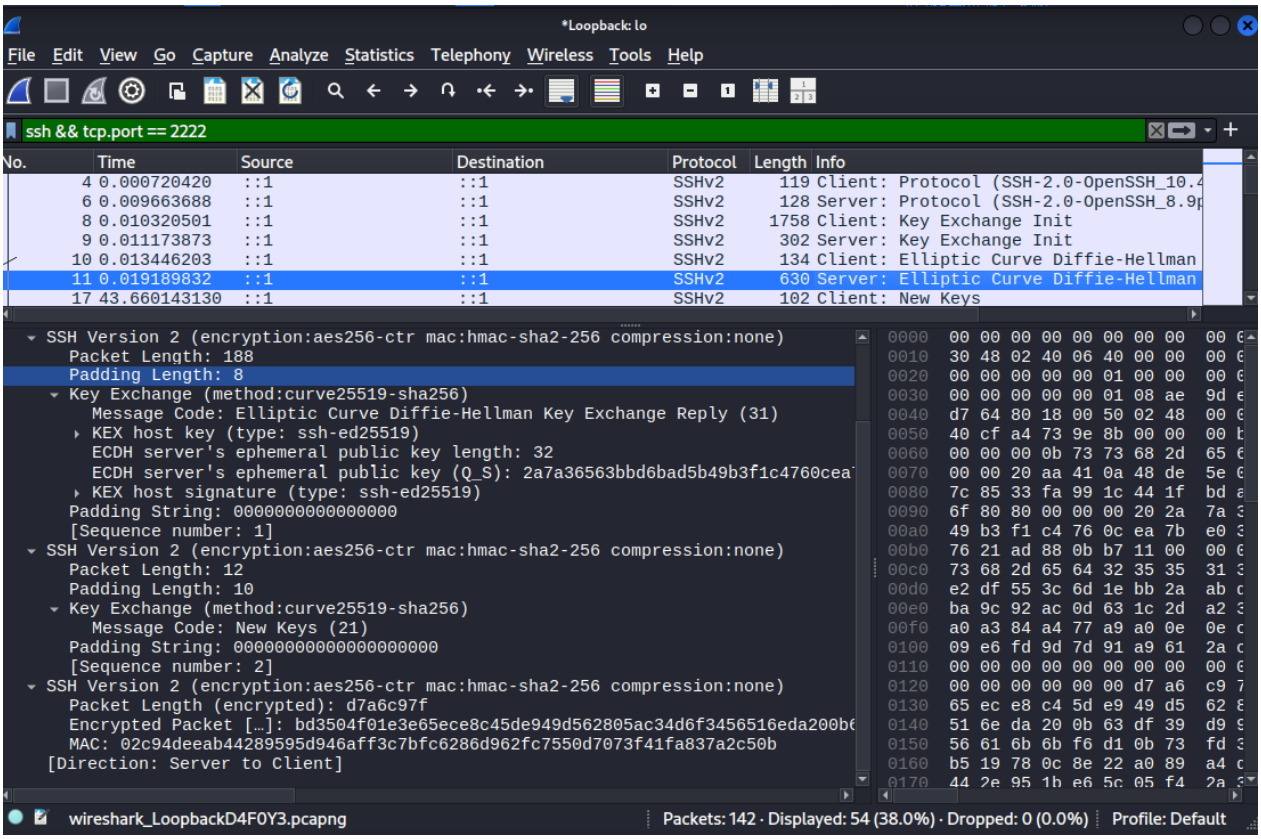


Figura 12: Paquete ECDH Key Exchange Reply del servidor

Después del mensaje New Keys todo va cifrado, por lo que no se ve el usuario, la contraseña ni los comandos ejecutados. Sin embargo, sí se pueden observar los siguientes elementos:

1. **Direcciones IP, puertos y metadatos:** se ve quién se conecta, a qué servidor y por qué puerto. En este caso las direcciones de origen y destino son :1 (la dirección de loopback de IPv6, ya que el servidor está en el mismo equipo) y el puerto es el 2222. También se aprecian el tamaño de los paquetes y los tiempos entre ellos, lo que permite estimar la duración de la sesión y cuándo hubo actividad (Figura 10).

2. **Versión del protocolo y del software:** en los primeros paquetes aparece en texto claro el banner con la versión de SSH y de OpenSSH del cliente y del servidor (en este caso OpenSSH 10.4p1 Debian en el cliente y OpenSSH 8.9p1 Ubuntu en el servidor, Figura 10). Esto podría ayudar a un atacante a buscar vulnerabilidades conocidas de esa versión.

3. **Algoritmos criptográficos ofrecidos:** en el paquete Key Exchange Init se ven los algoritmos que acepta el servidor (Figura 11), porque en ese momento todavía no existe una clave compartida para cifrarlos. Aquí se puede comprobar que solo se ofrecen los cuatro configurados.

4. **Claves públicas y firma:** en el paquete ECDH Key Exchange Reply se ven la clave pública efímera del servidor, su clave de host y la firma del intercambio (Figura 12). Esto no es un riesgo, ya que con las claves públicas no se puede calcular la clave secreta de la sesión.

3.2. Actividad 2

3.2.1. Identificación del paquete RSA_PIN

La captura entregada, `transacciones_cifradas.pcap`, contiene 41 paquetes TCP que van desde la IP 10.10.10.10 hacia la IP 10.10.10.20 al puerto 4444.

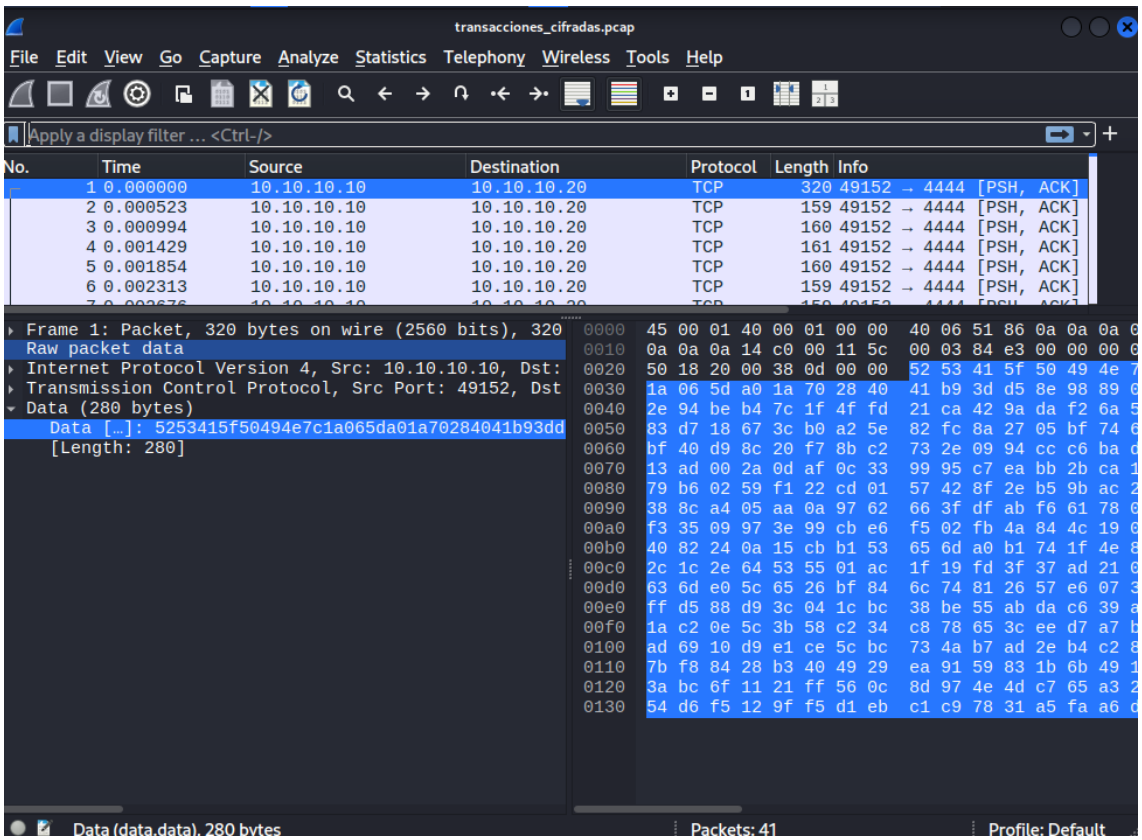


Figura 13: Paquetes de la captura transacciones cifradas.pcap

El paquete RSA PIN es el primero de la captura. Su payload es de 280 bytes y contiene la etiqueta RSA PIN—, luego la salt de 16 bytes y finalmente el PIN cifrado con RSA (256 bytes).

3.2.2. Identificación de los paquetes AESGCM

Los 40 paquetes restantes, es decir, los frames 2 al 41, contienen las transacciones. Cada uno tiene la etiqueta AESGCM—, un nonce de 12 bytes, el mensaje cifrado y un tag de

autenticación de 16 bytes al final.

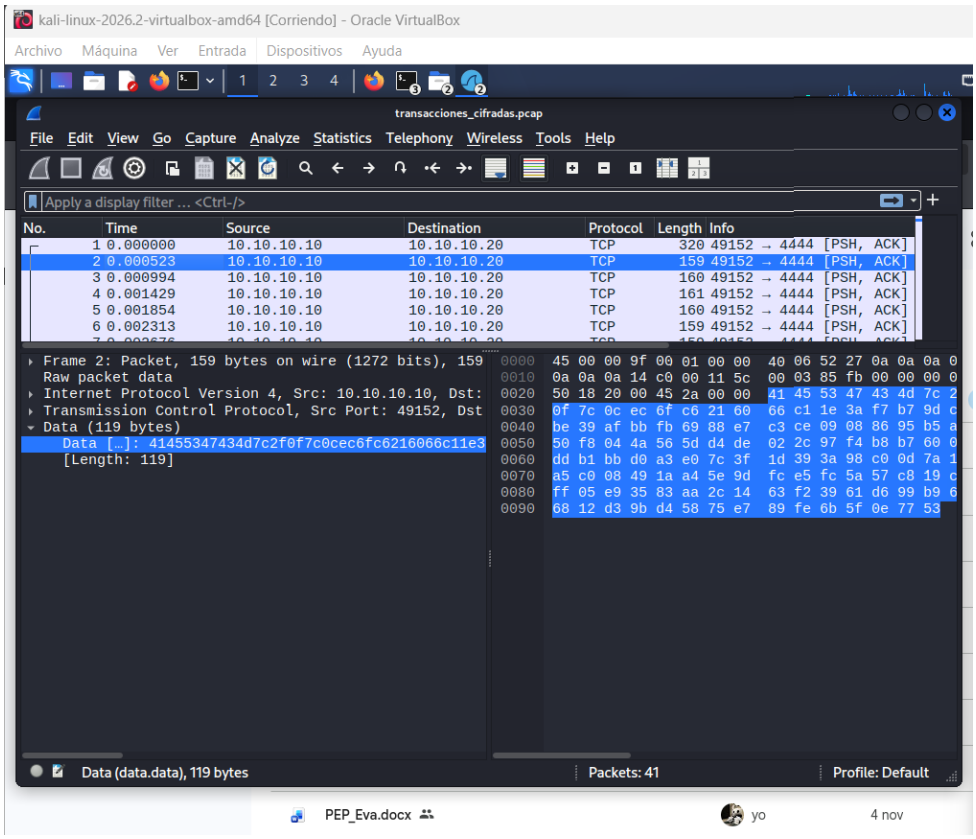


Figura 14: Paquete RSA PIN (frame 1).

Para demostrar que efectivamente lo hice yo se puede ver que estoy en el drive con mi correo y mi foto de usuario.

3.2.3. Recuperación del PIN y derivación de la clave AES

Se generó un script en Python que lee la captura, extrae la salt y el PIN cifrado del primer paquete y descifra el PIN con la clave privada RSA usando OAEP con SHA-256. El PIN recuperado es 4827. Luego, con el PIN y la salt, se derivó la clave AES de 256 bits usando Scrypt con $n = 2^{14}$, $r = 8$ y $p = 1$. El código completo se encuentra en el github.

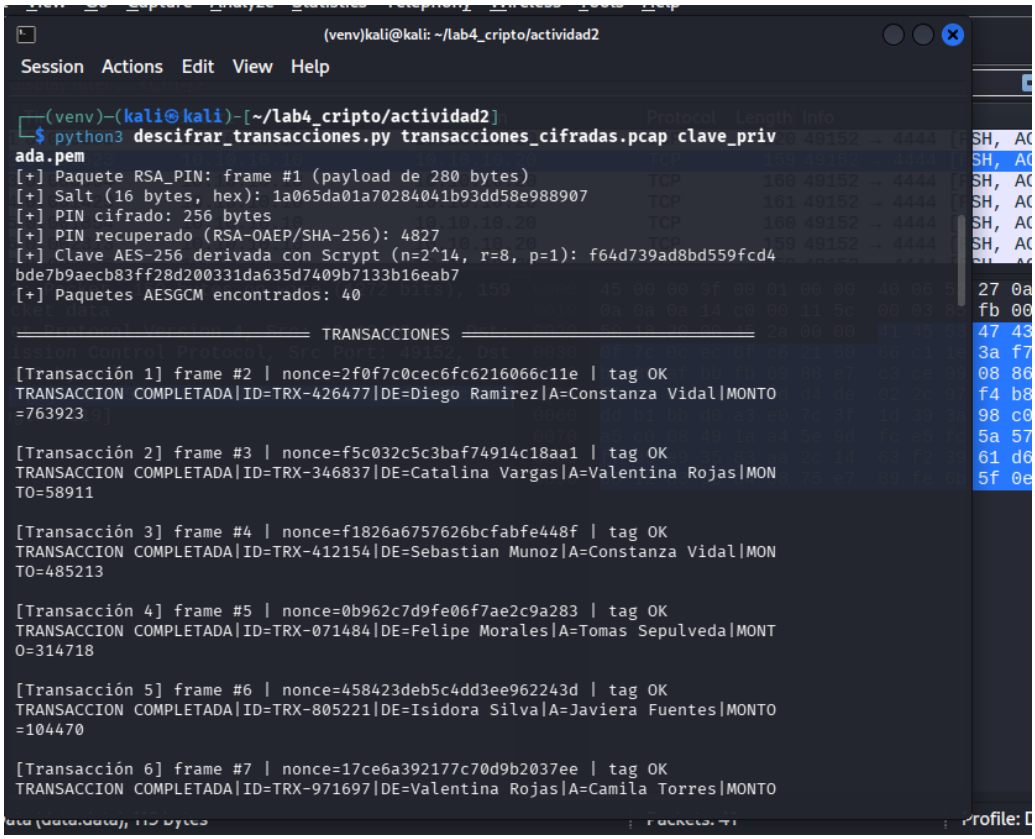


Figura 15: PIN recuperado, clave AES derivada y primeras transacciones (1 a 6).

Cabe aclarar de la salt que es un valor aleatorio, que no es secreto, y que se combina con el PIN para generar la clave. Hace que un mismo PIN produzca claves distintas y evita que se usen tablas con claves ya calculadas. Esto es importante porque un PIN de cuatro dígitos tiene solo 10.000 combinaciones posibles, así que sin salt sería muy fácil precalcular todas las claves.

En cuanto a los parámetros de Scrypt, estos son:

- $n = 2^{14}$: factor de costo, define cuánta memoria y cálculo requiere generar la clave.
- $r = 8$: tamaño de los bloques con que trabaja internamente, junto con n determina la memoria usada.
- $p = 1$: cantidad de cálculos en paralelo.

Estos parámetros hacen que derivar una clave sea lento a propósito, lo que dificulta probar muchos PIN por fuerza bruta.

También siempre hay que usar los mismos parámetros, porque Scrypt es determinista: siempre da el mismo resultado si recibe las mismas entradas. Si se cambia la salt o cualquiera de los parámetros (n, r o p), la clave obtenida sería distinta y no se podrían descifrar los mensajes.

3.2.4. Descifrar los mensajes

Con la clave AES, el script recorre los 40 paquetes AESGCM. De cada uno toma los primeros 12 bytes como nonce y descifra el resto con AES-GCM. El nonce debe ser distinto en cada mensaje, y el tag de 16 bytes permite verificar la integridad y autenticidad: al descifrar, AES-GCM revisa el tag y, si el mensaje hubiera sido modificado, mostraría un error. Las 40 transacciones se descifraron y verificaron correctamente, mostrando su ID, remitente, destinatario y monto. Las transacciones 1 a 6 se muestran en la Figura 15 y el resto a continuación.

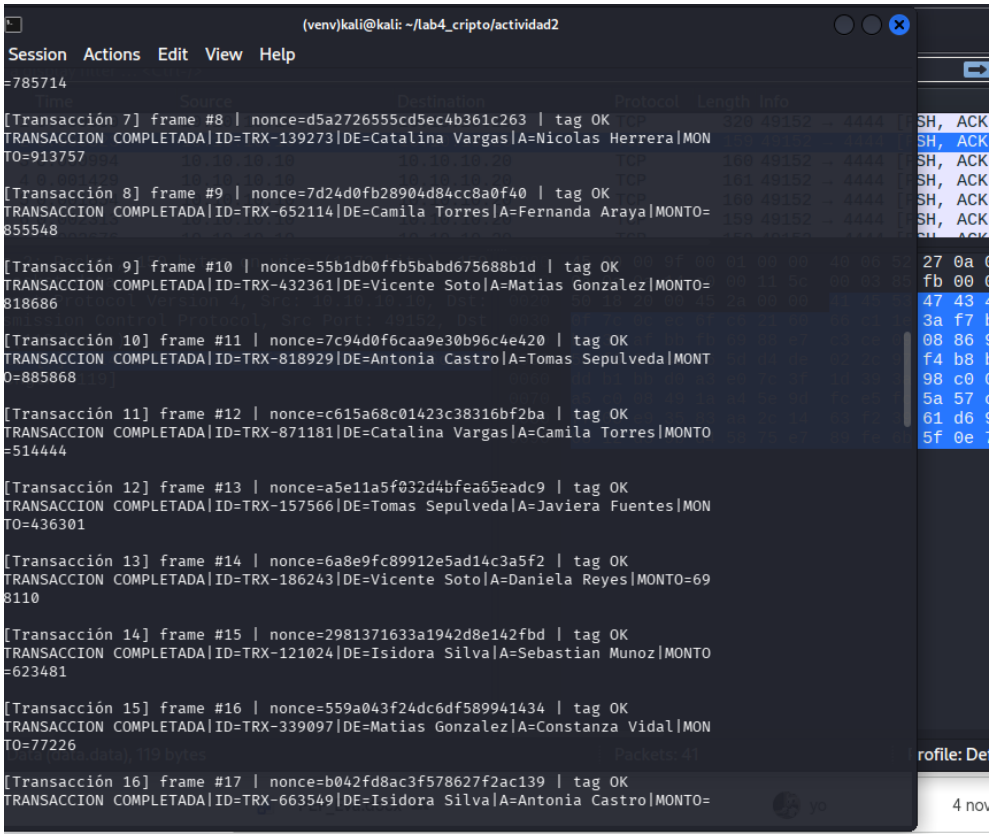


Figura 16: Transacciones 7 a 16.

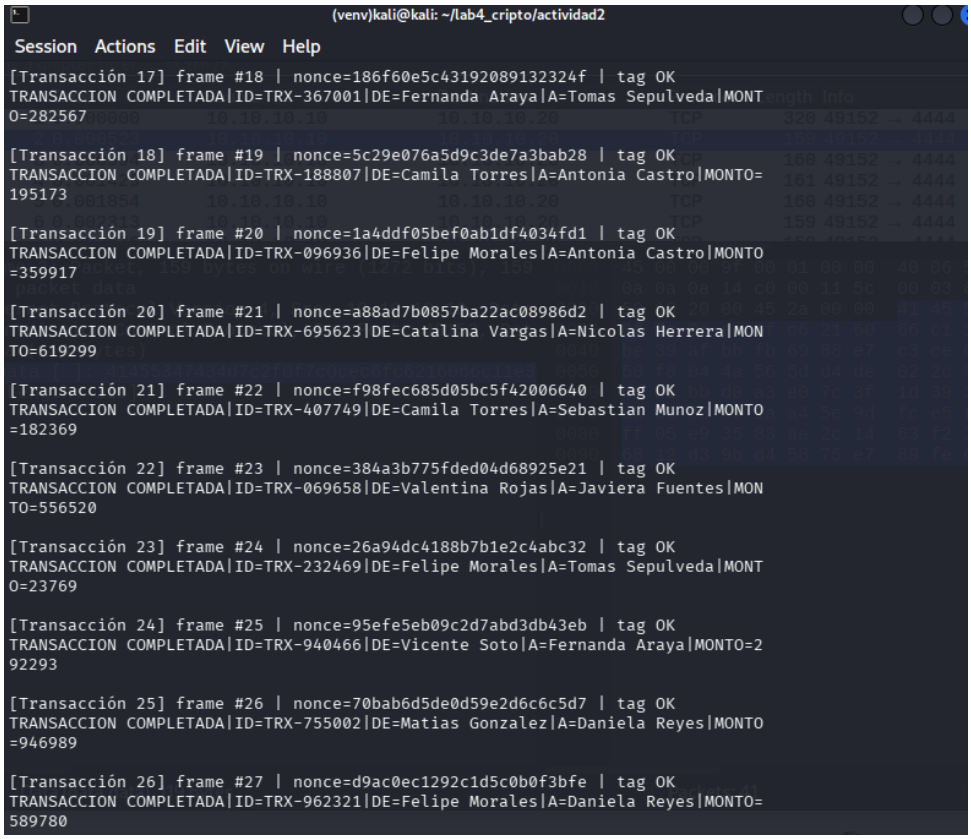


Figura 17: Transacciones 17 a 26.

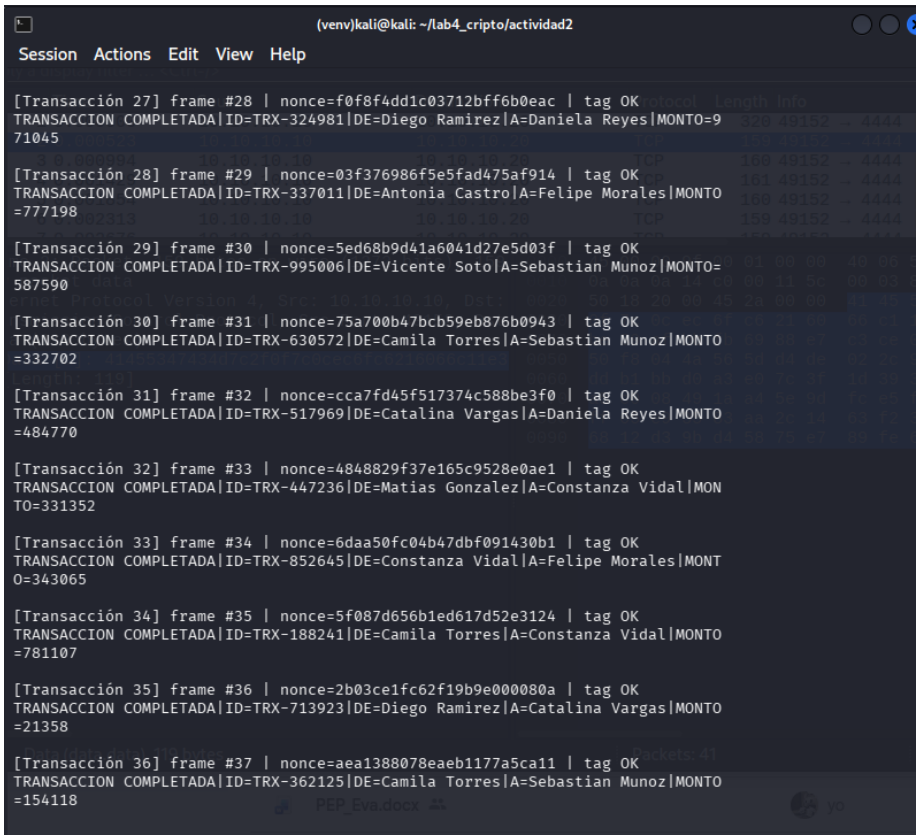


Figura 18: Transacciones 27 a 36.

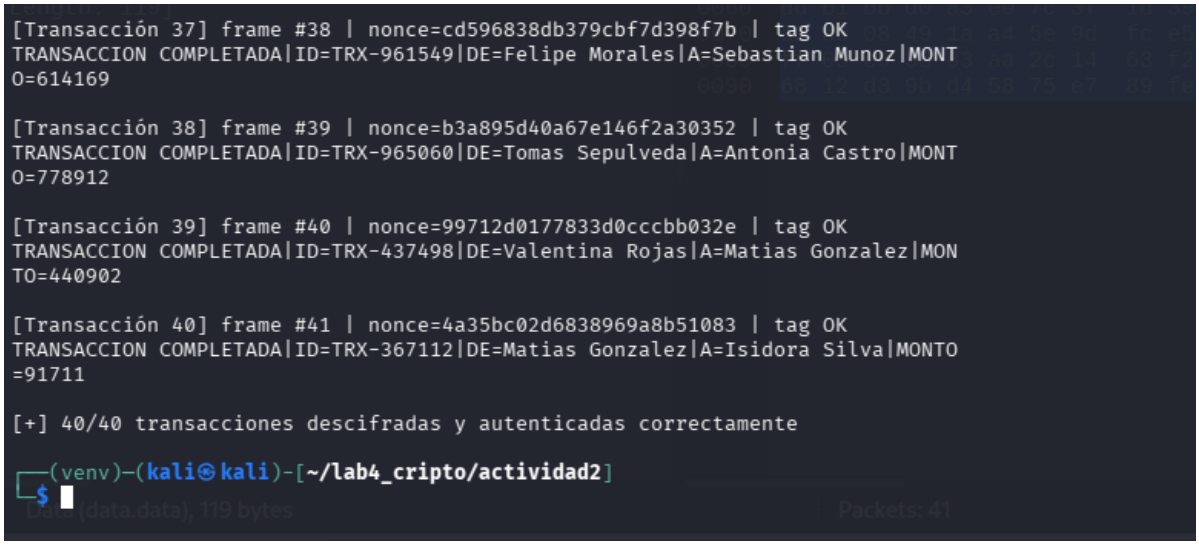


Figura 19: Transacciones 37 a 40 y verificación final (40/40 autenticadas).

Cabe mencionar que este esquema es híbrido: se usa RSA para proteger solo el PIN (un

dato pequeño, ya que RSA es lento y limitado en tamaño) y AES-GCM para proteger el contenido de las transacciones, que es más eficiente para grandes volúmenes de datos.

3.2.5. Análisis de criptografía postcuántica (PQC)

La respuesta a la pregunta ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC? es no. Se justifica por lo siguiente:

1. **Problemas matemáticos distintos:** RSA basa su seguridad en la dificultad de factorizar números enteros grandes, mientras que los estándares PQC, como ML-KEM (FIPS 203), se basan en problemas de reticulados (lattices). Un par de claves RSA no tiene ninguna relación con las claves de un algoritmo PQC.

2. **Formato de claves incompatible:** una clave privada RSA está compuesta por valores como d, N, p y q, mientras que un algoritmo PQC de reticulados espera como clave privada vectores de polinomios con coeficientes pequeños. No se pueden intercambiar.

3. **Paradigma de operación distinto:** en este laboratorio RSA-OAEP se usó para cifrar directamente un dato elegido (el PIN), pero los estándares PQC funcionan como mecanismos de encapsulación de claves (KEM), donde se genera un secreto compartido aleatorio y no se cifra un mensaje preexistente.

4. **Aclaración importante:** RSA es justamente el algoritmo vulnerable al algoritmo de Shor en un computador cuántico. Por eso, un atacante podría guardar este tráfico hoy y romper RSA en el futuro (*harvest now, decrypt later*), pero eso es romper RSA, no usar claves RSA para descifrar mensajes PQC.

Conclusiones y comentarios

En la Actividad 1 se comprobó que SSH protege el contenido de la sesión (usuario, contraseña y comandos), aunque deja visibles datos como las direcciones IP, la versión del software y los algoritmos negociados. En la Actividad 2 se recuperó el PIN con RSA, se derivó la clave AES con Scrypt y se descifraron y verificaron las 40 transacciones, lo que muestra cómo un esquema híbrido combina la comodidad de la criptografía asimétrica con la eficiencia de la simétrica. Por último, se concluyó que las claves RSA no sirven para descifrar nada protegido con PQC, aunque sí es un recordatorio de que el tráfico cifrado con RSA es vulnerable a futuro frente a computadores cuánticos.

4. Bibliografia

- Captura y Claves

- Ylonen, T. y Lonvick, C. (2006). *RFC 4253: The Secure Shell (SSH) Transport Layer Protocol*. IETF.

- Moriarty, K. et al. (2016). *RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2*. IETF.
- Percival, C. y Josefsson, S. (2016). *RFC 7914: The scrypt Password-Based Key Derivation Function*. IETF.
- Dworkin, M. (2007). *NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST.
- National Institute of Standards and Technology (2024). *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. `https://csrc.nist.gov/pubs/fips/203/final`